



沈阳建筑大学

上机实验报告

课程名称:	编译原理
实验题目:	实验四 语义分析
专业班级:	计算机 2101 班
姓 名:	张清晨
学 号:	2104230414
日 期:	2024.05.18

【实验目的】

上机目的：

通过上机实习，加深对语法制导翻译原理的理解，掌握将语法分析所识别的语法范畴变换为某种中间代码的语义翻译方法。

内容：

- (1) 选用目前普遍采用的语义分析方法—语法制导翻译技术；
- (2) 语义分析对象重点考虑经过语法分析后已是正确的语法范畴，实习重点是语义子程序；
- (3) 中间代码选用比较常见的形式，例如四元式、逆波兰式。

【基本原理和上机步骤】

基本原理：

语法制导翻译的基本思想：为每个产生式配上一个语义子程序，该子程序描述了一个产生式所对应的翻译工作。这些工作包括：生成中间代码，查填有关的符号表，检查和报错，修改编译程序某些工作变量的值等。在语法分析过程中，每当一个产生式用于匹配式进行归约时，就调用该产生式所对应的语义子程序，以完成既定的翻译任务。

主要的中间代码有：逆波兰、四元式、三元式、简介三元式、树

上机步骤：

中缀表达式转后缀表达式（逆波兰式）步骤：

1. 初始化两个栈：一个用于运算符的栈(S1)和一个用于操作数(如数字)的栈(S2)。
2. 从左到右扫描中缀表达式的每个字符。
3. 遇到操作数（如数字）时，直接将其压入操作数栈(S2)。
4. 遇到运算符时，比较其与运算符栈(S1)栈顶运算符的优先级：如果运算符栈(S1)为空，或栈顶运算符为左括号“（”，或者优先级比栈顶运算符的优先级较高，则直接将此运算符压入栈中。
5. 否则，将运算符栈(S1)中栈顶的运算符弹出并压到操作数栈(S2)，再次进行与运算符栈(S1)栈顶运算符的优先级比较。
6. 遇到括号时，如果遇到了左括号“（”，则直接压入运算符栈(S1)。如果遇到右括号“）”，则依次弹出运算符栈(S1)栈顶的运算符，并压入操作数栈(S2)，直到遇到左括号“（”为止，此时将这一对括号丢弃。
7. 重复上述步骤直到表达式的最右边。

8. 将运算符栈(S1)剩余的运算符依次弹出并压入操作数栈(S2)。
9. 拼接操作数栈(S2)中的元素并输出, 结果即为中缀表达式所对应的后缀表达式。

逆波兰式计算的步骤:

1. 初始化两个栈: 一个用于运算符的栈(S1)和一个用于操作数(如数字)的栈(S2)。
2. 从左到右扫描中缀表达式的每个字符。
3. 遇到操作数(如数字)时, 直接将其压入操作数栈(S2)。
4. 遇到运算符时, 比较其与运算符栈(S1)栈顶运算符的优先级: 如果运算符栈(S1)为空, 或栈顶运算符为左括号“(”, 或者优先级比栈顶运算符的优先级较高, 则直接将此运算符压入栈中。
5. 否则, 将运算符栈(S1)中栈顶的运算符弹出并压到操作数栈(S2), 再次进行与运算符栈(S1)栈顶运算符的优先级比较。
6. 遇到括号时, 如果遇到了左括号“(”, 则直接压入运算符栈(S1)。如果遇到右括号“)”, 则依次弹出运算符栈(S1)栈顶的运算符, 并压入操作数栈(S2), 直到遇到左括号“(”为止, 此时将这一对括号丢弃。
7. 重复上述步骤直到表达式的最右边。
8. 将运算符栈(S1)剩余的运算符依次弹出并压入操作数栈(S2)。
9. 拼接操作数栈(S2)中的元素并输出, 结果即为中缀表达式所对应的后缀表达式。

【实验结果】

输出的格式如下:

- (1) 输入一以#结束的中缀表达式(包括+—*/ () 数字#): 在此位置输入符号串如
(28+68)*2#
- (2) 逆波兰式为: 28&68&+&2&*&#
- (3) 逆波兰式 28&68+2*计算结果为 192

运行结果为:

```
E:\编译原理\上机实验\实验4\1 X
输入表达式：
(28+68)*2#
中缀表达式是：(28+68)*2#

逆波兰式是：&28&68&+&2&*#

最后结果：192.000000
请按任意键继续. . . |
```

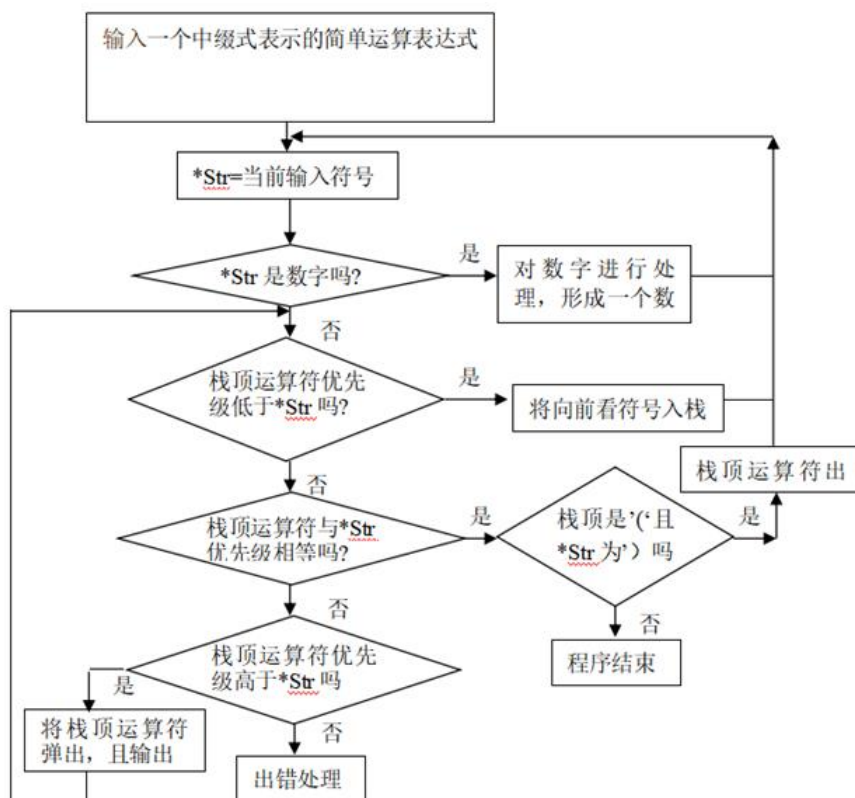
```
E:\编译原理\上机实验\实验4\1 X
输入表达式：
23+35*6#
中缀表达式是：23+35*6#

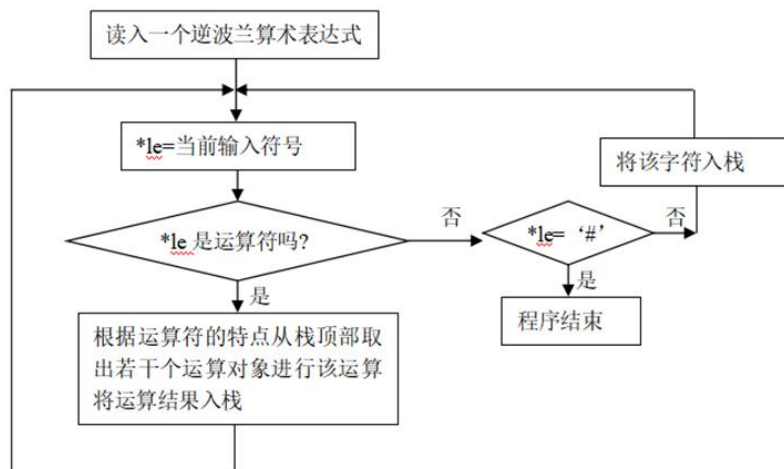
逆波兰式是：23&35&6&*&+&#

最后结果：233.000000
请按任意键继续. . . |
```

【讨论与分析】

实验逻辑图





实验过程记录: 在实验的过程中,我有很多次错误,首先我通过自己调试代码的方式解决,如果未能解决,又去上网查询资料,最后都能解决。

实验总结: 在此次语义分析实验中,我深刻感受到了将理论与实践相结合的必要性。通过对语法树的细致探究,我掌握了如何运用语义规则来验证程序逻辑的正确性。这一过程不仅锻炼了我的逻辑思维,也提升了我的编程技术。我深刻理解到,语义分析不仅是一项技术挑战,更是对编程语言深层次含义的探究。通过这次实验,我对编译器的工作机制有了更直观的了解,同时也增强了我解决复杂数题的信心。

【源程序代码】

```

#define _CRT_SECURE_NO_WARNINGS
#include "cstdio"
#include "string.h"
#include "stdlib.h"
#include "iostream"
#include "cstring"
#define MAXLEN 100
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2

using namespace std;

```

```

typedef struct
{
    char op;
    int level;
}opt;

typedef struct          //定义操作符栈
{
    opt st[MAXLEN];
    int top;

}op_stack;

typedef struct          //定义值栈
{
    double D[MAXLEN];
    int top;
}D_stack;

//-----对栈操作的定义-----
opt peek(op_stack* s)    //定义看栈顶函数
{
    opt error = { '$',-2 };
    if (s->top >= 0)
        return s->st[s->top];
    else
        return error;
}

int IsEmpty(op_stack* s)    //定义判断栈空的函数
{
    if (s->top < 0)
        return 0;
    else
        return s->st[s->top].op;
}

```

```

char push(op_stack* s, opt c)      //定义入栈函数
{
    s->top++;
    s->st[s->top] = c;
    return c.op;
}

opt pop(op_stack* s)              //定义出栈函数
{
    opt i;
    opt error = { '$', -2 };
    if (s->top >= 0)
    {
        i = s->st[s->top];
        s->st[s->top].op = '\0';
        s->top--;
        return i;
    }
    else
        return error;
}

void clear(op_stack* s)           //定义初始化栈
{
    s->top = -1;
}

//-----define the value stack-----
double Dpeek(D_stack* s)          //定义看栈顶函数
{
    if (s->top >= 0)
        return s->D[s->top];
    else
        return 0;
}

int DIsEmpty(D_stack* s)          //定义判断栈空的函数
{

```

```

        if (s->top < 0)
            return 0;
        else
            return (int)(s->D[s->top]);
    }

double Dpush(D_stack* s, double c)    //定义入栈函数
{
    s->top++;
    s->D[s->top] = c;
    return c;
}

double Dpop(D_stack* s)    //定义出栈函数
{
    double i;
    if (s->top >= 0)
    {
        i = s->D[s->top];
        s->D[s->top] = '\0';
        s->top--;
        return i;
    }
    else return 0;
}

void Dclear(D_stack* s)    //定义初始化栈
{
    s->top = -1;
}

//-----定义栈的操作完成-----
//-----中缀变逆波兰式的函数-----
void change(char* str, char* expr, char* kkk)
{
    opt A = { '+', 1 };
    opt R = { '-', 1 };

```



```

opt M = { '*',2 };
opt D = { '/',2 };
opt B = { '(',-1 };
opt Mo = { '%',2 };

op_stack os;
clear(&os);
while (*str != '#')
{

    while (*str >= '0' && *str <= '9' || *str == '.')
    {
        *expr = *str;
        expr++;
        str++;
    }

    *expr++ = '&';

    switch (*str)
    {
    case '+':
        while (peek(&os).level >= A.level && IsEmpty(&os))
        {
            *expr++ = pop(&os).op;
        }
        push(&os, A);
        str++;
        break;
    case '-':
        while (peek(&os).level >= R.level && IsEmpty(&os))
        {
            *expr++ = pop(&os).op;
        }
        push(&os, R);
    }
}

```

```

        str++;
        break;
    case '*':
        while (peek(&os).level >= M.level && IsEmpty(&os))
        {
            *expr++ = pop(&os).op;

        }
        push(&os, M);
        str++;
        break;
    case '/':
        while (peek(&os).level >= D.level && IsEmpty(&os))
        {
            *expr++ = pop(&os).op;

        }
        push(&os, D);
        str++;
        break;
    case ':':
        push(&os, B);
        str++;
        break;
    case ')':
        while (peek(&os).level != -1)
        {
            *expr++ = pop(&os).op;
        }
        pop(&os);
        str++;
        break;
    case '#':
        while (IsEmpty(&os))
        {
            *expr++ = pop(&os).op;
            *expr++ = '&';//must add this line
        }
        *expr--;

```

```

        *expr = '#';
        break;
    case '%':
        while (peek(&os).level >= Mo.level)
        {
            *expr++ = pop(&os).op;
        }
        push(&os, Mo);
        str++;
        break;
    }
}

//-----后缀求值函数-----
double calval(char* le)
{
    int i, leng;
    double result = 0;
    D_stack Data;
    double tmpData, tmp1;
    char tmp[MAXLEN] = { '\0' };
    int a = 0;
    Dclear(&Data);
    while (*le != '\0')
    {
        while (*le >= '0' && *le <= '9' || *le == '.')
        {
            tmp[a++] = *le++;
        }
        le++;
        tmp[a] = '\0';
        tmp1 = atof(tmp);
        if (tmp1 != 0.0) {
            Dpush(&Data, tmp1);
            leng = strlen(tmp);
            for (i = 0; i < leng; i++)    //重新初始化临时数组
                tmp[i] = '\0';
            a = 0;
        }
    }
    result = tmp1;
}

```

```

    }
    switch (*le)
    {
    case '&':
        le++;
        break;
    case '#':
        le++;
        break;
    case '+':
        result = Dpop(&Data) + Dpop(&Data);
        Dpush(&Data, result);
        le++;
        break;
    case '-':
        result = Dpop(&Data) - Dpop(&Data);
        Dpush(&Data, result);
        le++;
        break;
    case '*':
        result = Dpop(&Data) * Dpop(&Data);
        Dpush(&Data, result);
        le++;
        break;
    case '/':
        result = Dpop(&Data) / Dpop(&Data);
        Dpush(&Data, result);
        le++;
        break;
    case '%':
        tmpData = Dpop(&Data);
        result = (double)((int)Dpop(&Data) % (int)tmpData);
        Dpush(&Data, result);
        le++;
        break;
    }
}

```

```

return result;

```

```

}
//-----主函数-----

int main()
{
    char string[MAXLEN];
    char exp[MAXLEN] = { '\0' };
    //char exp[MAXLEN] = "1#2#*3#4#*#+";
    //-----数据定义结束-----
    char* kkk;
    kkk = exp;
    puts("输入表达式:");
    gets(string);
    printf("中缀表达式是:%s\n\n", string);
    change(string, exp, kkk);

    printf("逆波兰式是:%s\n", exp);

    printf("\n 最后结果:%f\n", calval(exp));

    system("PAUSE");

    return 0;
}

```